

Chapter 07 · Part 1

Runnables in LangChain — Part 1

Why LangChain's components were not built to talk to each other, and how the Runnable interface was introduced to standardize every component into one common, composable contract.

WHAT THIS CHAPTER COVERS

- ▶ Why Runnables exist: the historical problem
- ▶ The "too many chains" crisis
- ▶ Non-standardized components (before)
- ▶ What a Runnable actually is
- ▶ The Lego-block analogy
- ▶ Building a Runnable abstract class
- ▶ RunnableConnector: composing runnables
- ▶ Chains of chains (recursive composition)

Runnables in LangChain — Part 1

RECAP — CHAPTER 06 · CHAINS

The previous chapter covered **Chains** — LangChain's built-in mechanism for linking two or more components together into a pipeline. Three chain types were built: **Sequential Chains** (output of one LLM call feeds the next), **Parallel Chains** (multiple LLM calls run independently on the same input and are combined), and **Conditional Chains** (the pipeline branches based on a classification result). This chapter goes one level deeper and explains the internal mechanism — **Runnables** — that makes Chains possible in the first place.

1. Why Runnables Exist — The Historical Context

Definition

Runnables are the standardized building-block interface that every LangChain component (LLMs, prompt templates, retrievers, parsers, chains) is built on top of. They are the internal mechanism that allows components to be connected to one another with zero custom glue code.

Why It Exists

To understand Runnables, it helps to trace how LangChain evolved:

- 1 **Late 2022:** ChatGPT is released and OpenAI opens its APIs to the public. Developers can now build their own LLM-based applications on top of OpenAI's models.
- 2 **Multiple LLM providers emerge:** OpenAI, Anthropic, Google, Mistral, and others each expose their own APIs, and each API behaves differently. The LangChain team's first goal was to build a framework that let developers talk to *any* provider's LLM through one consistent class, with minimal code changes.
- 3 **Beyond the LLM call:** The team noticed that talking to an LLM is only one small piece of building a real application. A PDF-reader application, for example, also needs to load a document, split it into chunks, generate embeddings, store them in a vector database, retrieve relevant chunks, and finally format the answer.
- 4 **Components are born:** To help with all of this — not just the LLM call — LangChain introduced dedicated components: Document Loaders, Text Splitters, Embedding Models, Vector Stores, Retrievers, Output Parsers, and Memory. Developers could now pick up these components like building blocks and assemble any kind of LLM application.

LLM

PromptTemplate

Chain

Retriever

Parser

Runnable

RunnableConnector

Example — Two Applications Built From Components

A simple LLM application (ask a topic, generate a poem) needs only an LLM and a `PromptTemplate` component. A far more complex RAG-based PDF reader (load → split → embed → store → retrieve → answer) could be written in roughly 30–36 lines of code once these components existed — a task that would have taken far more manual code without them.

🔄 Quick Revision — Historical Context

- LangChain first solved the problem of talking to many different LLM providers through one consistent interface.
- It then broke every step of building an LLM application into its own dedicated component (loaders, splitters, embedders, retrievers, parsers).
- These components let engineers assemble complex LLM applications (like RAG pipelines) with dramatically less code.

2. The Problem: Too Many Chains

How It Works (Or Rather, How It Broke)

Once components existed, the LangChain team noticed a pattern: certain sequences of components (prompt → LLM, or retrieval → prompt → LLM) were reused across almost every application. To save developers from writing this glue code manually, they wrapped these common patterns into built-in functions called *Chains* — starting with **LLMChain** (prompt + LLM) and **RetrievalQAChain** (retriever + prompt + LLM).

This worked well at first, so the team kept extending the idea — adding a chain type for every new use case: `SimpleSequentialChain`, `SequentialChain`, `SQLDatabaseChain`, `APICChain`, `LLMMathChain`, and dozens more.

COMMON MISTAKE

It is tempting to think that *more built-in chains = more power*. In practice, LangChain's proliferation of chain types became a liability: a large, hard-to-maintain codebase, and a steep learning curve for new engineers who had to memorize which chain applied to which use case.

Root Cause

The real problem was not the number of chains — it was that LangChain's components (LLM, `PromptTemplate`, `Retriever`, `Parser`) were never designed with a shared interface. Each one had been built independently and exposed a different method for doing its job.

BEFORE — EVERY COMPONENT SPEAKS DIFFERENTLY



AFTER — ONE COMMON INTERFACE

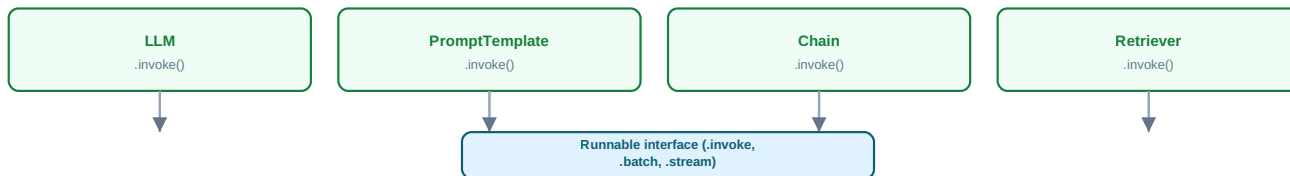


Figure 7.1 — Before standardization every component exposes a different method name; after standardization every component exposes the same `.invoke()` method.

Component	Old (non-standard) method
LLM	<code>.predict()</code>
PromptTemplate	<code>.format()</code>
Chain	<code>.run()</code>
Retriever	<code>.get_relevant_documents()</code>

Because none of these interfaces matched, connecting any two components together required hand-written custom code — which is exactly how `LLMChain` and `RetrievalQAChain` came to exist as one-off functions in the first place. Every new use case meant another one-off chain function, and the codebase kept growing.

Quick Revision — The Chains Problem

- LangChain's built-in Chains were created to automate common prompt → LLM (and retrieval → prompt → LLM) patterns.
- Over time the team kept adding a new chain type for every new use case, resulting in dozens of chain classes.
- This bloated the codebase and made the library harder to learn for new engineers.
- The root cause: components had incompatible interfaces (`.predict`, `.format`, `.run`, `.get_relevant_documents`), so connecting them always needed custom code.

3. Code Walkthrough — Components Without Standardization

How It Works

To feel the problem directly, the lecture builds a simplified, dummy version of LangChain's original (pre-Runnable) design: a `DummyLLM` with a `.predict()` method and a `DummyPromptTemplate` with a `.format()` method.

```
# Dummy LLM: mimics the old .predict() interface
import random

class DummyLLM:
    def __init__(self):
        return None

    def predict(self, prompt):
        response_list = [
            "Delhi is the Capital of India.",
            "IPL is a cricket league in India.",
            "AI stands for Artificial Intelligence.",
        ]
        return {"response": random.choice(response_list)}
```

```
# Dummy PromptTemplate: mimics the old .format() interface
class DummyPromptTemplate:
    def __init__(self, template, input_variables):
        self.template = template
        self.input_variables = input_variables

    def format(self, input_dict):
        return self.template.format(**input_dict)
```

Both classes work, but notice the two method names are different: `predict()` vs. `format()`. To chain them together, a third class is needed — a hand-built `DummyChain` that knows, by hard-coded convention, to call `.format()` first and then `.predict()`:

```
# DummyChain: manually written glue code (this is what LLMChain really was)
class DummyChain:
    def __init__(self, prompt, llm):
        self.prompt = prompt
        self.llm = llm

    def run(self, input_dict):
        formatted_prompt = self.prompt.format(input_dict)
        result = self.llm.predict(formatted_prompt)
        return result["response"]

llm = DummyLLM()
template = DummyPromptTemplate(
    template="Write a {length} poem about {topic}.",
    input_variables=["length", "topic"]
)
chain = DummyChain(template, llm)
output = chain.run({"length": "short", "topic": "the ocean"})
```

This works for exactly this one combination of components. The moment a new requirement appears — for example, a two-step chain that generates a joke and then explains it — `DummyChain.run()` has no way to support it, because it

was hard-coded to call exactly one `.format()` and one `.predict()`. A completely new custom class would be needed for every new workflow shape.

IMPORTANT

This is the core insight of the chapter: it is **not possible to build a flexible, general-purpose chaining mechanism** on top of components that each expose a different method name. The glue code itself has to be reinvented for every new combination.

🔄 Quick Revision — Non-Standardized Design

- DummyLLM exposes `.predict()`; DummyPromptTemplate exposes `.format()` — two different method names for "do your job".
- DummyChain hard-codes the exact sequence: call `.format()` then `.predict()`.
- This design cannot scale to new workflow shapes (e.g. a 2-step joke → explanation pipeline) without writing an entirely new custom class.
- This is exactly the real-world origin story of LangChain's LLMChain and RetrievalQAChain.

4. What Is a Runnable?

Definition

A *Runnable* is a standardized unit of work. Every Runnable in LangChain accepts an input, processes it, and returns an output — and, critically, every Runnable exposes the exact same set of method names to do so.

Core Characteristics

#	Characteristic	Meaning
1	Unit of work	Every Runnable has one clear purpose: take an input, produce an output.
2	Common interface	Every Runnable exposes the same method names — most importantly <code>.invoke()</code> , plus <code>.batch()</code> and <code>.stream()</code> .
3	Composable	Because the interface is identical, any two Runnables can be connected: the output of one becomes the input of the next.
4	Self-similar	A chain built by connecting Runnables is itself a Runnable, so it can be connected to other Runnables (or other chains) again.

REAL WORLD INSIGHT — THE LEGO-BLOCK ANALOGY

Runnables behave exactly like Lego blocks: each block is a unit of work, every block shares the same connector interface regardless of its shape, any two blocks can be snapped together, and the resulting combined structure is itself a valid “block” that can be connected further. This self-similarity is what allows arbitrarily complex LLM pipelines to be built from simple parts.

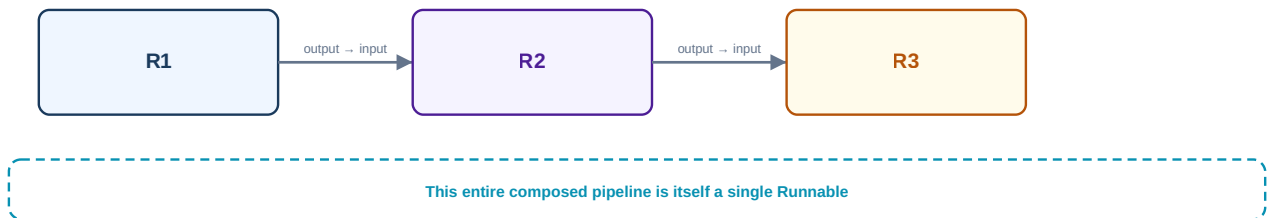


Figure 7.2 — Runnables connect output-to-input, and the resulting composed pipeline is itself a single Runnable.

How It Works — Building the Runnable Abstract Class

The standardization is implemented using **abstraction** from object-oriented programming. An abstract base class named `Runnable` is created with one abstract method, `invoke()`. Any component class that inherits from `Runnable` is then forced by Python to implement its own `invoke()` method, guaranteeing every component exposes the same method name.

```
from abc import ABC, abstractmethod

class Runnable(ABC):
    # invoke() does nothing here; every child class MUST implement it
    @abstractmethod
    def invoke(input_dict):
        pass
```

Trying to instantiate a class that inherits from `Runnable` without implementing `invoke()` raises `TypeError: Can't instantiate abstract class ... with abstract method invoke` — this is Python enforcing the contract.

```
# DummyLLM standardized: .predict() logic moves into .invoke()
import random

class DummyLLM(Runnable):
    def __init__(self):
        return None

    def invoke(self, prompt):
        response_list = [
            "Delhi is the Capital of India.",
            "IPL is a cricket league in India.",
            "AI stands for Artificial Intelligence.",
        ]
        return {"response": random.choice(response_list)}
```

```
# DummyPromptTemplate standardized: .format() logic moves into .invoke()
class DummyPromptTemplate(Runnable):
    def __init__(self, template, input_variables):
        self.template = template
        self.input_variables = input_variables

    def invoke(self, input_dict):
        return {"response": self.template.format(**input_dict)}
```

```
# A new standardized component: extracts the string from a response dict
class DummyStrOutputParser(Runnable):
    def __init__(self):
        pass

    def invoke(self, input_dict):
        return input_dict["response"]
```

BEST PRACTICE

When retiring an old method name like `.predict()` or `.format()` in favour of `.invoke()`, do not delete it immediately — older code that still calls the old method name would break. Keep the old method working, but print a deprecation warning telling engineers to migrate to `.invoke()`.

Quick Revision — The Runnable Interface

- A Runnable is an abstract unit of work: input in, output out, via a common method set.
- The core method every Runnable exposes is `.invoke()`; `.batch()` and `.stream()` also exist for other cases.
- Python's ABC + `@abstractmethod` is used to force every component subclass to implement `.invoke()`.
- Because every component now speaks `.invoke()`, any two Runnables can be connected without writing custom glue code.
- Deprecated methods (`.predict`, `.format`) should be kept temporarily with a warning, not deleted outright.

5. RunnableConnector — Composing Runnables Into Chains

Definition

`RunnableConnector` is a `Runnable` whose entire job is to take a list of other `Runnables` and execute them in sequence, automatically feeding the output of each one into the input of the next.

How It Works

Its constructor accepts a list of `Runnables`. Its `invoke()` method loops over that list, calling `.invoke()` on each `Runnable` and overwriting the working `input_data` variable with each step's output, so the next iteration automatically receives the previous step's result.

```
class RunnableConnector(Runnable):
    def __init__(self, runnable_list):
        self.runnable_list = runnable_list

    def invoke(self, input_data):
        for runnable in self.runnable_list:
            input_data = runnable.invoke(input_data)
        return input_data
```

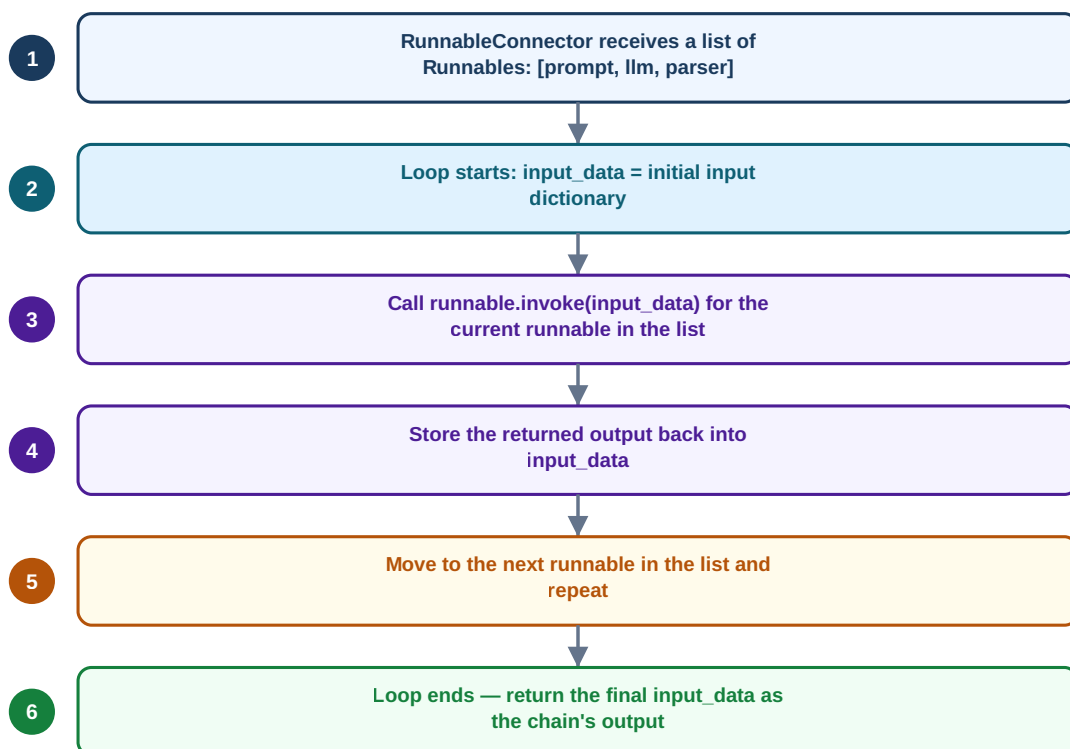


Figure 7.3 — `RunnableConnector` loops through its list of `Runnables`, threading each step's output into the next step's input.

Example — A Two-Component Chain

```
llm = DummyLLM()
prompt1 = DummyPromptTemplate(
    template="Write a joke on {topic}.",
    input_variables=["topic"]
)

chain1 = RunnableConnector([prompt1, llm])
result = chain1.invoke({"topic": "cricket"})
# prompt1.invoke() formats the prompt → its output feeds llm.invoke() automatically
```

A third component can be added to the same list with zero extra code changes — for example a `DummyStrOutputParser` at the end, so the final chain returns a clean string instead of a dictionary:

```
parser = DummyStrOutputParser()
chain2 = RunnableConnector([prompt1, llm, parser])
# Now the final output is a plain string, not {"response": "..."} 
```

INTERVIEW TIP

If asked “how does a LangChain Chain work internally?”, the strongest answer is: a Chain is a `RunnableSequence` under the hood — conceptually identical to this lecture's `RunnableConnector` — that loops through a list of `Runnables` and pipes each one's output into the next one's input, relying entirely on the fact that every component shares the same `.invoke()` interface.

Quick Revision — RunnableConnector

- `RunnableConnector` takes a list of `Runnables` in its constructor.
- Its `invoke()` method loops through the list, calling `.invoke()` on each item.
- Each step's output automatically becomes the next step's input — no manual formatting or predicting needed.
- Any number of standardized components can be added to the list (prompt, LLM, parser, ...) without writing new custom code.

6. Chains of Chains — Recursive Composition

Why It Matters

Because a `RunnableConnector` is itself a `Runnable` (it inherits from the same abstract class and implements `invoke()`), a chain built from `Runnables` can be treated as just another `Runnable` and connected into a bigger chain. This is the direct, practical payoff of the Lego-block property described in Section 4.

Example — Joke Generator + Joke Explainer

Two independent chains are built: `chain1` generates a joke about a topic, and `chain2` takes a joke and explains it.

```
prompt1 = DummyPromptTemplate(
    template="Write a joke about {topic}.",
    input_variables=["topic"]
)
prompt2 = DummyPromptTemplate(
    template="Explain the following joke {response}.",
    input_variables=["response"]
)
llm = DummyLLM()
parser = DummyStrOutputParser()

chain1 = RunnableConnector([prompt1, llm])
chain2 = RunnableConnector([prompt2, llm, parser])
```

Instead of invoking each chain manually and passing data between them by hand, both chains are simply passed into a new `RunnableConnector`:

```
final_chain = RunnableConnector([chain1, chain2])
final_result = final_chain.invoke({"topic": "Freedom"})
# chain1 runs fully → its dict output feeds directly into chain2
print(final_result) # a plain string: the joke's explanation
```

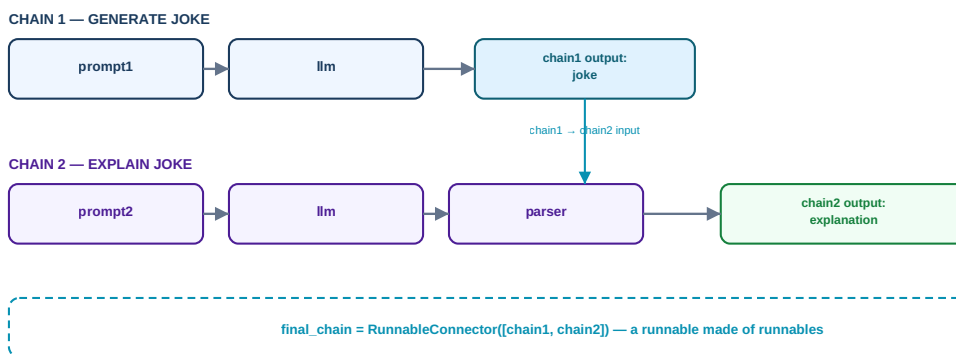


Figure 7.4 — `chain1` and `chain2` are each independent `Runnable` pipelines; because both are `Runnables`, they connect into `final_chain` exactly like individual components would.

IMPORTANT

Nothing new had to be built to support chaining chains together. `RunnableConnector` does not know or care whether the items in its list are individual components or entire sub-chains — because both satisfy the same `Runnable` contract, they are interchangeable from its point of view.

Quick Revision — Chains of Chains

- A `RunnableConnector` is itself a `Runnable`, since it also implements `.invoke()`.
- This means fully-built chains can be passed into another `RunnableConnector` exactly like individual components.
- `final_chain = RunnableConnector([chain1, chain2])` runs `chain1` to completion, then feeds its output into `chain2`.
- This recursive composability is what allows arbitrarily deep and complex LLM pipelines to be built from simple, reusable parts.

7. Confirming This In LangChain's Real Source Code

How It Works

The dummy classes built in this chapter are a simplified mirror of LangChain's actual internal design. Inspecting LangChain's real `ChatOpenAI` class shows an inheritance chain that leads back to the same idea:

```
ChatOpenAI
└→ BaseChatOpenAI
    └→ BaseChatModel
        └→ BaseLanguageModel
            └→ RunnableSerializable
                └→ Runnable    # the abstract base class, exactly like this chapter's Runnable
```

Scrolling through LangChain's real `Runnable` class confirms it defines an abstract `invoke()` method with no implementation — identical in concept to the `Runnable` abstract class built from scratch in Section 4. Every LangChain component you use (LLMs, chat models, prompt templates, retrievers, output parsers, and chains) ultimately inherits from this same `Runnable` base.

REAL WORLD INSIGHT

What was built in this chapter using two dummy classes and roughly 40 lines of code is exactly what LangChain does internally with a much larger, more feature-complete codebase. The underlying idea — one abstract class, one shared `invoke()` contract, unlimited composability — is identical.

Quick Revision — Real LangChain Verification

- `ChatOpenAI` inherits through `BaseChatOpenAI` → `BaseChatModel` → `BaseLanguageModel` → `RunnableSerializable` → `Runnable`.
- LangChain's real `Runnable` class defines an abstract `invoke()` method, just like the from-scratch version built in this chapter.
- Every major LangChain component (LLMs, prompts, retrievers, parsers, chains) ultimately inherits from this same `Runnable` base class.

K. Key Takeaways

- **Historical origin:** LangChain first standardized access to different LLM providers, then built dedicated components for every step of an LLM application (loaders, splitters, embedders, retrievers, parsers).
- **The Chains crisis:** To automate common component combinations, LangChain kept adding new built-in Chain classes for every new use case, leading to a bloated codebase and a steep learning curve.
- **Root cause:** Components were never designed with a shared interface — LLM used `.predict()`, PromptTemplate used `.format()`, Chain used `.run()`, Retriever used `.get_relevant_documents()`.
- **The fix — Runnable:** An abstract base class with one contract, `.invoke()`, that every component is forced to implement, making every component interchangeable and composable.
- **RunnableConnector:** A Runnable that loops through a list of Runnables, threading each step's output into the next step's input, with zero custom glue code.
- **Chains of chains:** Because a RunnableConnector is itself a Runnable, fully-built chains can be composed into bigger chains recursively, just like individual components.
- **Real-world mirror:** LangChain's actual ChatOpenAI class inherits down to the same Runnable abstract base, confirming this chapter's from-scratch implementation matches the real library design.

R. Revision Sheet

Runnable

An abstract base class defining one contract — `.invoke()` — that every LangChain component must implement, making all components interchangeable and composable.

Why Standardization Was Needed

Before Runnables, every component exposed a different method name (`.predict`, `.format`, `.run`), forcing custom glue code (and a new Chain class) for every new combination of components.

RunnableConnector

A Runnable that accepts a list of Runnables and executes them in sequence, automatically piping each step's output into the next step's input.

Self-Similarity / Lego-Block Property

A chain composed of Runnables is itself a Runnable, so chains can be nested inside other chains to build arbitrarily complex pipelines.

Deprecation Practice

Old method names (like `.predict()`) should be kept temporarily with a warning message rather than removed immediately, to avoid breaking existing code.

Real LangChain Mirror

`ChatOpenAI` → `BaseChatOpenAI` → `BaseChatModel` → `BaseLanguageModel` → `RunnableSerializable` → `Runnable` is the real inheritance chain confirming this design.

I. Interview Questions

Q1. Why did the LangChain team originally introduce built-in Chain classes like `LLMChain` and `RetrievalQChain`?

Q2. What two major problems arose from LangChain having a very large number of specialized Chain classes?

Q3. What is a Runnable, and what is the single most important method every Runnable must implement?

Q4. How does Python's `abc` module and the `@abstractmethod` decorator enforce that every component implements `.invoke()`?

Q5. Explain, with an example, how `RunnableConnector` threads the output of one Runnable into the input of the next.

Q6. Why is it possible to pass a fully-built chain into another `RunnableConnector`, just like an individual component?

Q7. What is the recommended practice when deprecating an old method name (e.g. `.predict()`) in favour of a new standardized one (`.invoke()`)?

Q8. Describe the Lego-block analogy for Runnables and explain which four properties of Runnables it illustrates.

F. Further Reading

- LangChain Python documentation — Runnable interface and LangChain Expression Language (LCEL)
- LangChain source code — `langchain_core.runnables.base.Runnable`
- Python official documentation — `abc` module and abstract base classes
- LangChain source code — `ChatOpenAI` and its inheritance chain through `BaseChatModel` / `BaseLanguageModel`